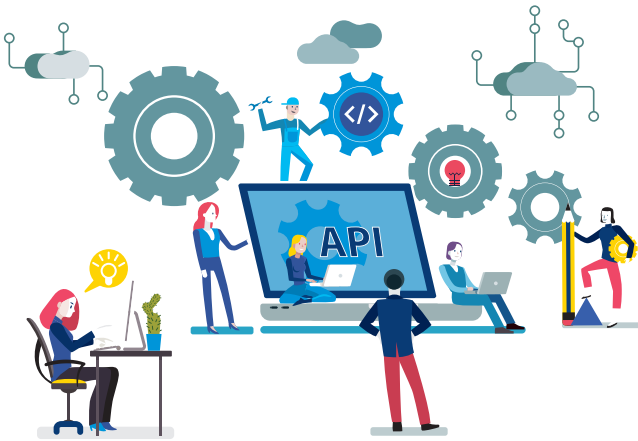


The Journey Towards Microservices



Designing applications around microservices architecture decomposes a monolith into a set of smaller and collaborative services. The icing on the cake is that we can evolve and scale each of the microservices independently of each other. This transition leverages the strong foundation laid by the object-oriented and domain-driven design patterns. This fifth part of 'The Design Odyssey' series demonstrates the evolution of a user management system (UMS) into a bunch of microservices.

For many, microservices are associated with Web applications, cloud services or at least REST API. Though there is a basis for such an impression, microservices need not always be designed in that way.

What is a microservice?

The basic premise of a microservice is that (in the words of Jeff Bezos of Amazon): “A team that can be fed with just two pizzas should be able to handle a microservice.”

A monolith is a relatively large and complex application that is developed and deployed as just one unit, as shown in Figure 1. It offers multiple functionalities and that's why it tends to be huge, heavy and rigid. For example, it may be in the form of a single large WAR deployed on a Tomcat Web server. When the load on the server increases, the performance of the monolith falls. Also, adding new features to a monolith is a daunting and time-taking task.

When a monolith is decomposed into a set of smaller microservices, they evolve independently of each other. The microservices are usually aligned with the lines of business of the organisation. Each microservice covers only a specific business

requirement. It corresponds to the concept of a sub-domain in the domain-driven design. It will have a domain model specific to its sub-domain. As we all know, a smaller domain model is easy to understand, develop and deploy.

Each team can choose the best platform suitable to develop its specific microservice; yet the services at runtime collaborate with each other to deliver the business value. New services can be added easily and deprecated services can be decommissioned effortlessly.

These services may be exposed via a REST interface, messaging interface or any other interface to the Web clients, mobile clients, IoT clients or standalone clients.

When the microservices are containerised, orchestrated and deployed on cloud infrastructure, they can easily be replicated to scale horizontally!

Service decomposition

The first step in the journey towards microservices is to decompose the monolith. A careful inspection of the monolith gives us an idea of how to peel the slices of functionality from it and convert them into independent microservices.

The user management service (UMS) that we developed as a monolith in the previous parts of this series offers three primary functionalities: 1) To add users to the system; 2) To find a specific user in the system; and 3) To search for users based on some criteria. Apart from these, the UMS is also logging the user-addition events in a central place.

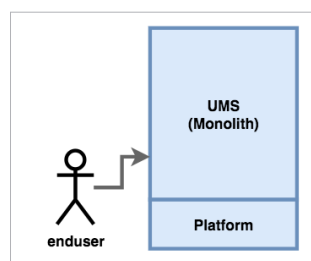


Figure 1: UMS as a monolith